

Planning and Learning with Tabular Methods

Reinforcement learning – LM Artificial Intelligence
(2025-26)

Alberto Castellini
University of Verona

References

- R. S. Sutton, A. G. Barto. Reinforcement learning, An Introduction. Second edition. Chapter 8

Summary

- Models and Planning
- Dyna: Integrated Planning, Acting and Learning
- Dyna-Q
- Example: Dyna Maze
- When the model is wrong
- Prioritized Sweeping
- Trajectory Planning

Models and Planning

- **Goal:** to develop an **unified view** of
 - RL **methods** that require a **model of the environment** (e.g., dynamic programming, heuristic search)
→ **Model-based RL methods**
 - RL **methods** that can be used **without a model** (e.g., MC, TD)
→ **Model-free RL methods**
- **Model-based methods:** rely on **planning**
- **Model-free methods:** rely on **learning**

Similarities:

- The hearth of both methods is the **computation of value functions**
- All methods are based on
 - 1) **looking ahead** to future events
 - 2) Computing a **backed up** value
 - 3) **Update** the approximate value function

Model of the environment: anything that the agent can use to **predict** how the environment will respond to its actions.

$$S_t, A_t \rightarrow \text{Model} \rightarrow S_{t+1}, R_{t+1}$$

- If the model is **stochastic** there are several possible next states and rewards with different **probabilities** (the opposite is **deterministic**)
- **Distribution models:** produce a description of **all possibilities** and related **probabilities**
- **Sample models:** produce just **one of the possibilities** according to probabilities
- Example: sum of a dozen dice
- **DP** uses a distribution model $p(s', r \mid s, a)$; in **blackjack** example we used a sample model

Models and Planning

- **Distribution models** are **stronger** than sample models (they can be used to generate samples)
- **Sample models** are usually **easier** to generate
- E.g., in **dozen dice** example it is
 - **easy** to write a program that simulates dice rolls and computes the sum,
 - **harder** to figure out all possible sums and related probabilities
- Given a **starting state** and a **policy**:
 - **Sample model**: produces a complete **episode**
 - **Distribution model**: produces **all** possible episodes/probabilities
- The **model** is used to **simulate the environment** and **produce simulated experience**

Models and Planning

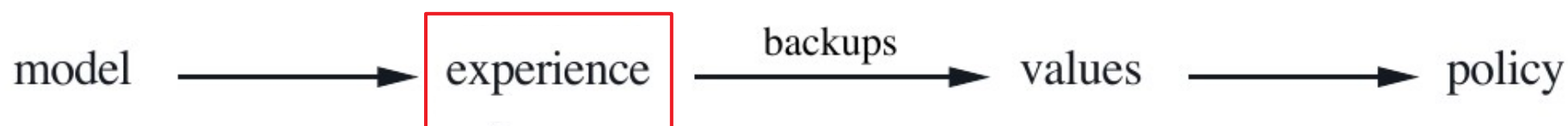
- **Planning**: computational process that takes a **model** as input and produces or improves a **policy**



- **Two approaches** to planning:
 - **State-space planning**: search through the state space for an optimal **policy** or an optimal path to goal. **Actions** cause transitions between states. **Value functions** computed over states
 - **Plan-space planning**: search through the space of plans. **Operators** transform plans to other plans. **Value functions** are defined over the space of plans (e.g., evolutionary methods, partial-order planning)
- **Problem: Plan-space methods are difficult** to apply efficiently to stochastic sequential decision problems (i.e., the target of RL)
- We focus on state-space planning

Models and Planning

- **State-space planning** methods and **learning** methods share a **common structure**
- Two basic ideas:
 - 1) All state-space planning methods involve computing **value functions** as an intermediate step toward improving the **policy**
 - 2) They compute **value functions** by updates of **backup operations applied to some experience**



- **DP**
 - i) makes **sweeps** through the state space
 - ii) generates for each state the distribution of **possible transitions**
 - iii) uses it to compute a **backed-up value** (update target)
 - iv) **update** the state's estimated value

Models and Planning

- **Similarity:** both learning and planning methods estimate **value functions** by **backing-up update** operations
- **Difference:** planning uses **simulated experience**, learning uses **real experience**
- Ideas and algorithms can be **transferred** between planning and learning
- Example: **Learning methods** in many cases can be applied to **simulated experience**

Example: **Random-sample one-step tabular Q-learning** converges to the **optimal policy for the model** as one-step tabular Q-learning converges to the optimal policy for the **real environment**

Random-sample one-step tabular Q-planning

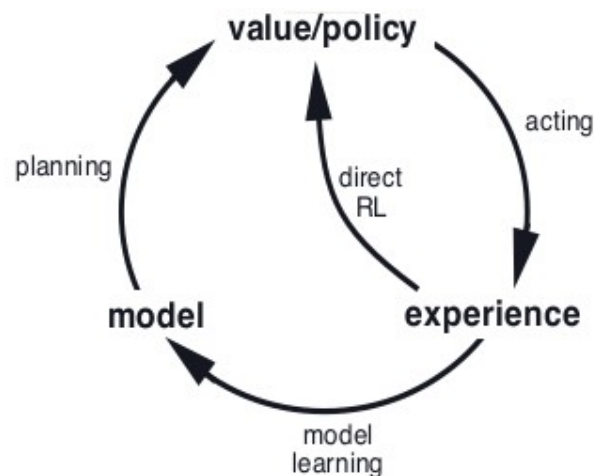
Loop forever:

1. Select a state, $S \in \mathcal{S}$, and an action, $A \in \mathcal{A}(S)$, at random
2. Send S, A to a sample model, and obtain
a sample next reward, R , and a sample next state, S'
3. Apply one-step tabular Q-learning to S, A, R, S' :
$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

Dyna: Integrated Planning, Acting and Learning

- When **planning** is done **online**, information gained from the interaction with the environment may be used to **improve the model** used by planning itself
- **Dyna-Q** is a **simple architecture** integrating the **major functions** needed in an **online planning agent**
- **Two roles for real experience** in online planners:
 - 1) **Improve the model** (i.e., make it more similar to the real environment) → **Model learning**
 - 2) **Improve the value function and policy** → **Direct RL**

Relationships between **experience**, **model**, **values** and **policy**



- **Experience** can improve **value function** and **policy** either **directly** or **indirectly** (via the **model**) → **Direct/Indirect RL**
- **Indirect methods: make fuller use of a limited amount of experience.** Better policy with fewer environment interactions. Cognition, deliberative planning.
- **Direct methods: simpler and not affected by biases** in the design of the **model**. Trial-and-error learning, reactive decision making

Dyna-Q

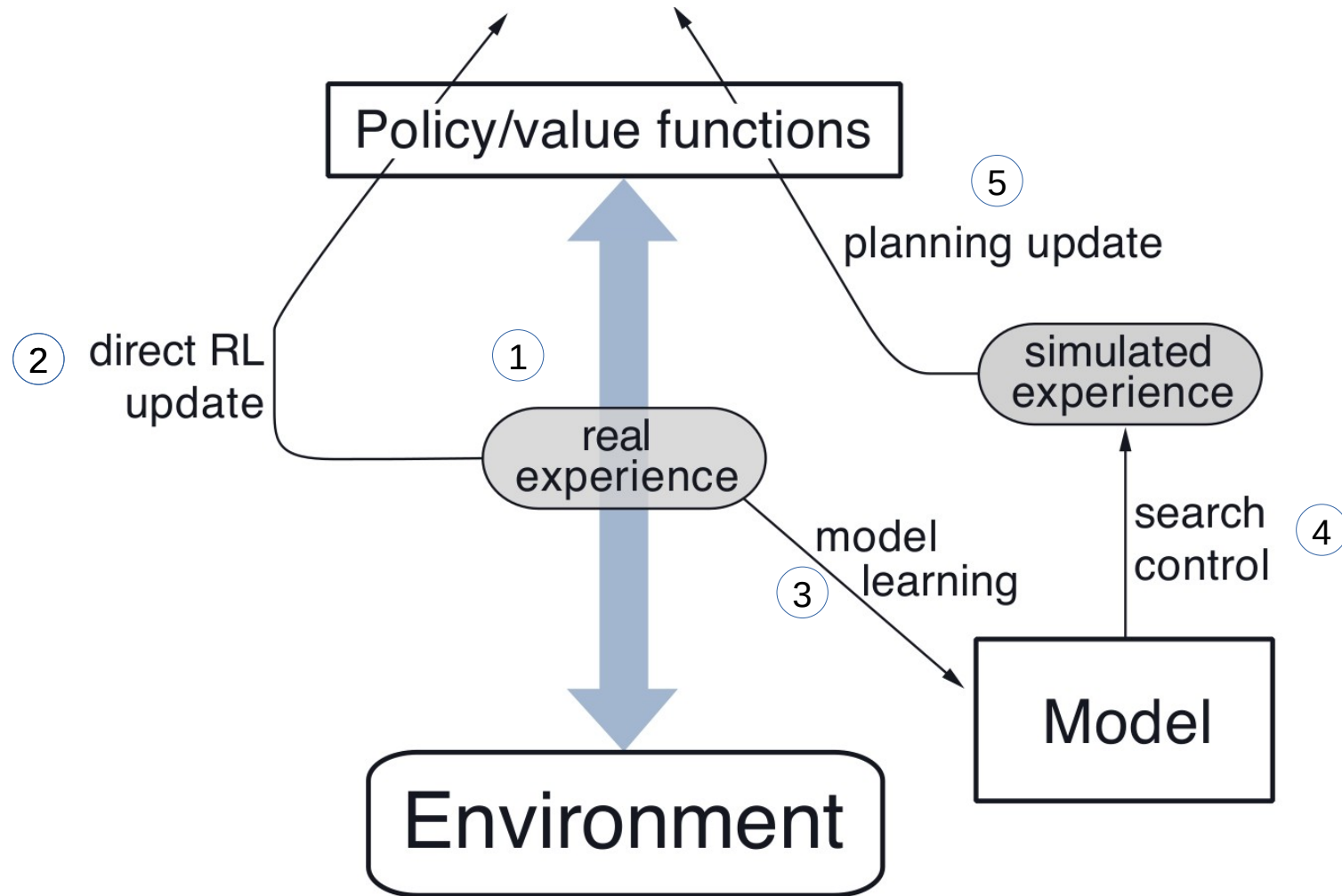
DynaQ includes:

- **Planning**: random-sample one-step tabular Q-learning
- **Direct RL**: one-step tabular Q-learning
- **Model learning**: **table** based with **deterministic environment**.
 - After each **transition** $S_t, A_t \rightarrow R_{t+1}, S_{t+1}$
the **model records** in its **table entry** for S_t, A_t the prediction that R_{t+1}, S_{t+1} will probabilistically follow

If the model is **queried again** with the same state-action pair it **returns the last-observed next-state and next reward** as its prediction

The **Q-planning algorithm** samples only from **state-action pairs** that have **previously** been **experienced**

Dyna architecture



- **DynaQ** is an example of this architecture
- **Learning** and **planning** differ in the **source of their experience**

Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

(a) $S \leftarrow$ current (nonterminal) state

(b) $A \leftarrow \epsilon$ -greedy(S, Q)

(c) Take action A ; observe resultant reward, R , and state, S'

(d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

(e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)

(f) Loop repeat n times:

$S \leftarrow$ random previously observed state

$A \leftarrow$ random action previously taken in S

$R, S' \leftarrow Model(S, A)$

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

In DynaQ

- Acting, model-learning and direct RL require little computation
- **Planning**: is **computationally-intensive**

Algorithm:

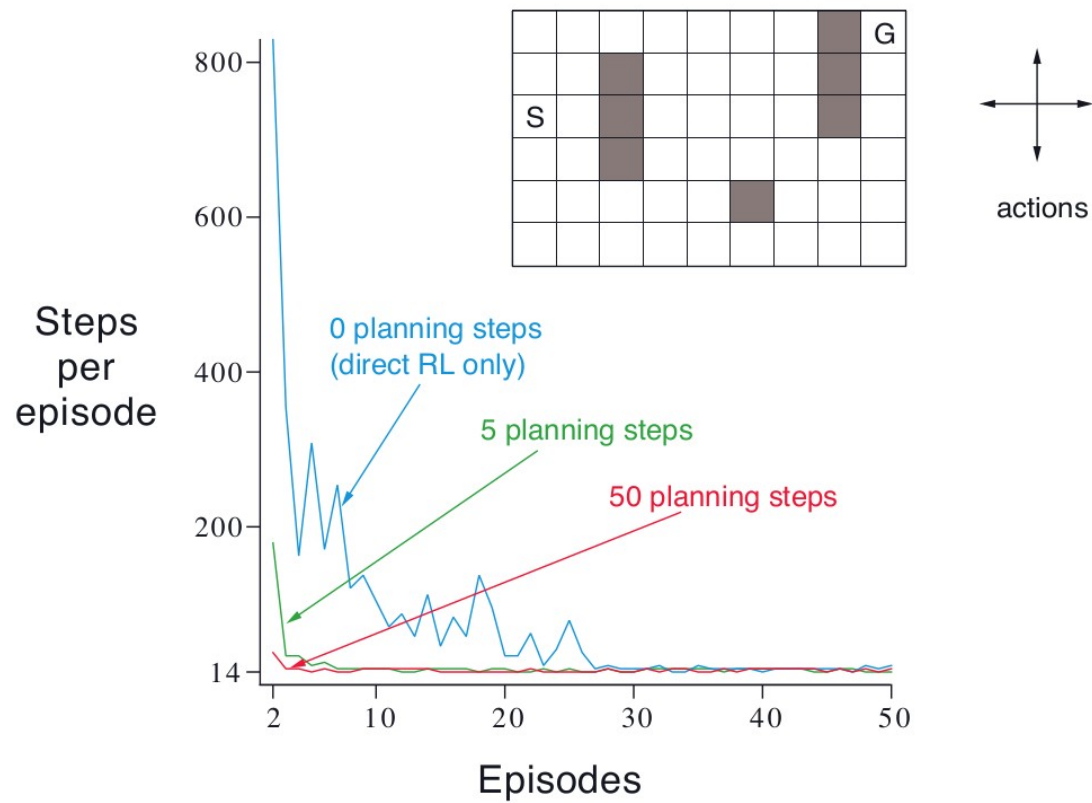
- Step d: **direct RL**
- Step e: **model learning** (according to real experience)
- Step f: **planning**
- If steps e and f are omitted then we obtain one-step tabular Q-learning
- In planning $Model(s,a)$ is the content (r,s) for state-action (s,a)

Notice:

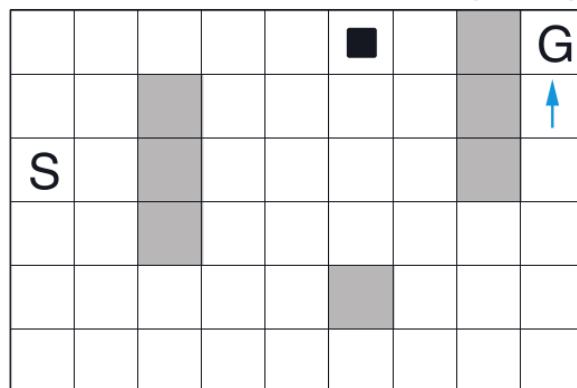
- planning proceeds incrementally, hence it is **trivial to intermix planning and acting**
- The agent is always both reactive and deliberative

Example: Dyna Maze

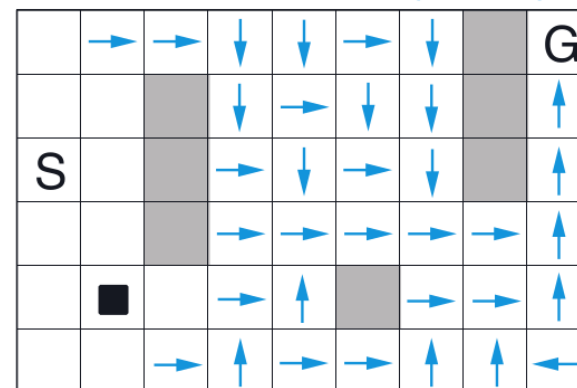
Dyna Maze



WITHOUT PLANNING ($n=0$)



WITH PLANNING ($n=50$)



When the model is wrong

When the model is wrong

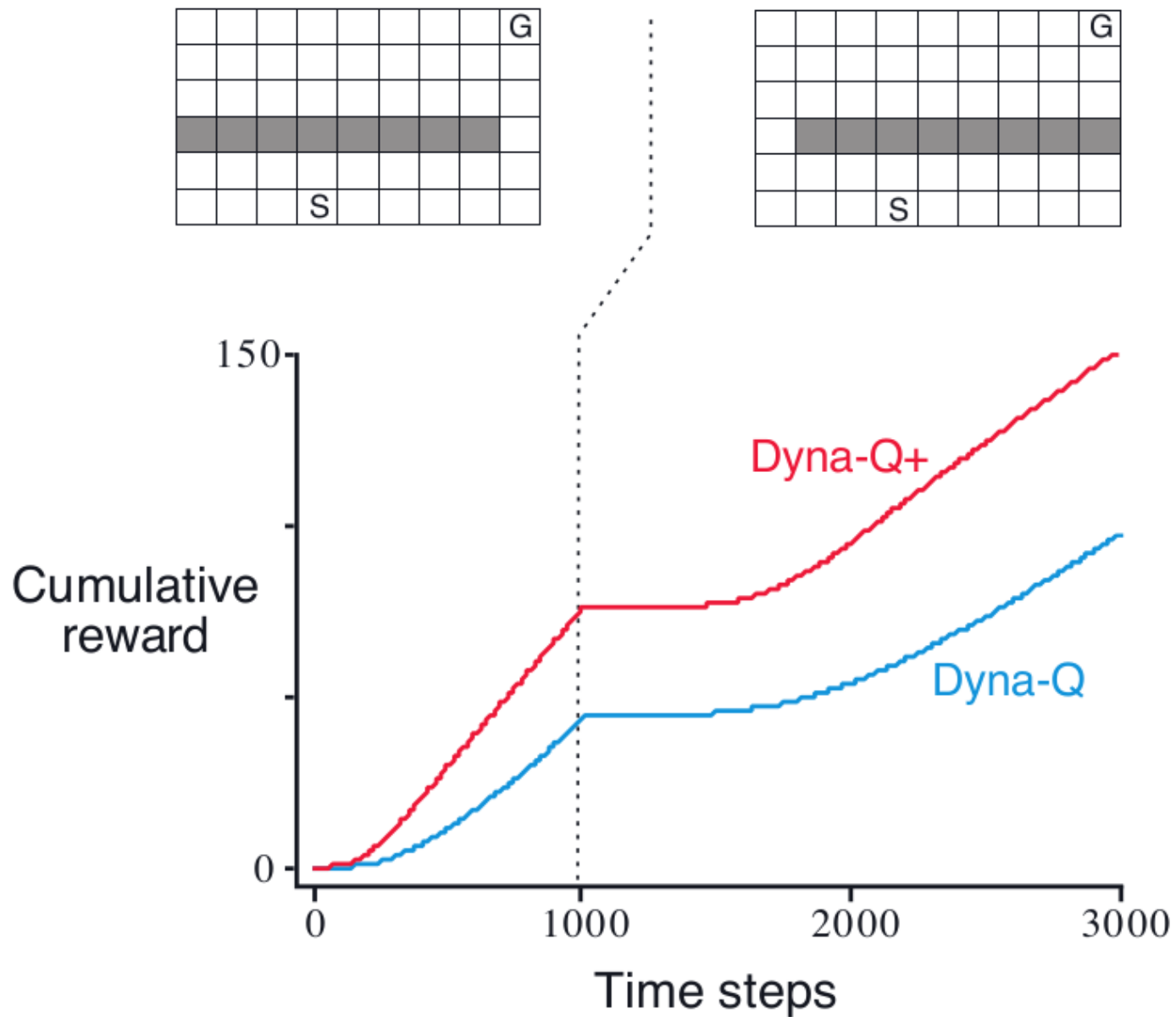
Models may be incorrect because:

- The **environment is stochastic** and only a limited number of samples have been observed
- The model was learned using **function approximation** that has generalized imperfectly
- The **environment has changed** and its new behavior has not yet been observed

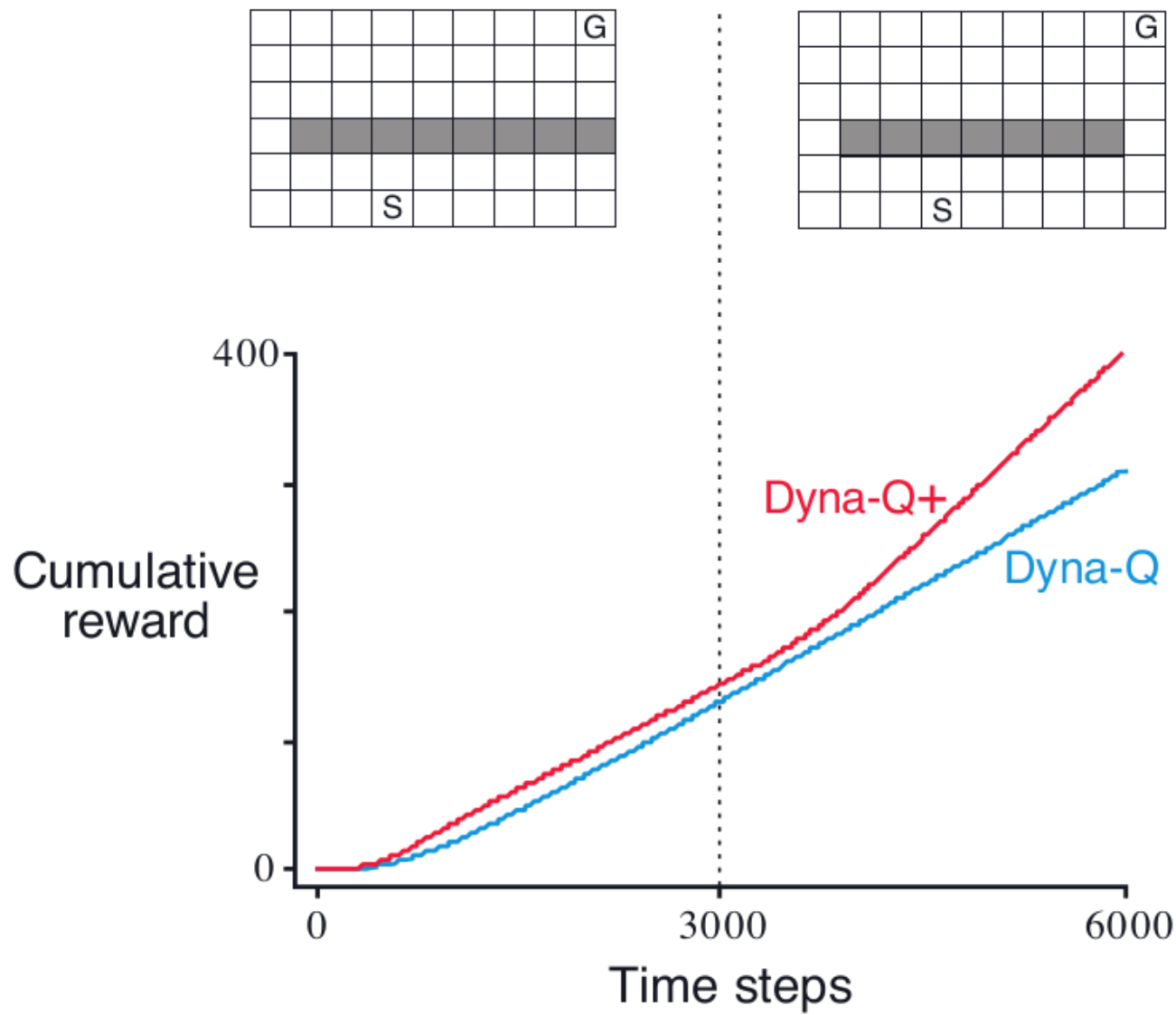
When the model is incorrect the planning process is likely to compute a **suboptimal policy**

In cases of **optimistic model** (predicting greater reward or better state transitions than possible) the suboptimal policy may lead to the **discovery and correction** of the **modeling error**

Example: blocking maze (case of optimistic model)



Example: shortcut maze (case of pessimistic model)



When the model is wrong

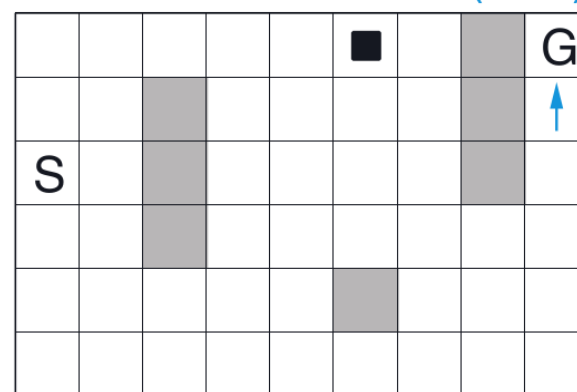
- General problem: another version of the **conflict** between **exploration and exploitation**
- In planning
 - **Exploration** = trying **actions** that **improve the model**
 - **Exploitation** = **behaving optimally** given the **current model**
- We want the agent to **explore** to find **changes in the environment** but **not so much** that performance is greatly degraded
- **No solution** but simple **heuristics**
- E.g., **DynaQ+** for **shortcut maze**: keep track for **each state-action pair** of **how many time steps have elapsed since the pair was last tried in real interaction with the environment**
 - The **more time elapsed**, the **greater the chance the model is incorrect**
 - **Bonus reward on simulated experiences involving these actions**

Prioritized Sweeping

Prioritized Sweeping

- In Dyna, **simulated transitions** are started in **state-action pairs selected uniformly at random** from all previously experienced pairs
- **Planning can be more efficient if simulated transitions and updates are focused on particular state-action pairs**
- **E.g.**, second episode of **maze** task: only last s,a pair has positive value.
 - **Pointless to perform model updates along almost all trajectories** because they take the agent from one zero-valued state to another (**updates have no effect**)
- **As planning progresses, the region of useful updates grows**, but planning is still less efficient than if **focused** where it can achieve best improvements

WITHOUT PLANNING ($n=0$)



Prioritized Sweeping

- In **real-world domains** the number of **states** is so **large** that an unfocused search would be very inefficient
- **A first strategy:** the search might be focused by working **backward from goal states**
- **A more general strategy:** the search might be focused by **working backward from any state whose value has changed**
- **E.g.**, when the maze agent discovers a change (or an incorrect transition) in the environment it changes its estimated value of one state.
- The values of many other states must be changed accordingly but **the only useful one-step updates are those of actions that lead directly into the state whose value has been changed**
- This is said **Backward Focusing**

Prioritized Sweeping

- **Another improvement:** as the frontiers of useful updates propagate backward, it often grows rapidly.
But **not all** these state-action pairs are **equally useful**
- The values of some states may have **changed a lot**, others **little**
- **The predecessor pairs of those that have changed a lot are likely to also change a lot**
- In **stochastic environments**, variations in estimated transition **probabilities** also contribute to variations in the **size** of changes and in the **urgency** with which pairs need to be changed
- **Prioritized sweeping:** **prioritize the updates** according to a **measure** of their **urgency** and perform them in order of priority
 - **Queue of state-action pairs** whose estimated value would change non-trivially if updated, **prioritized by the size of the change** → Effects of changes are efficiently back propagated

Prioritized Sweeping

Prioritized sweeping for a deterministic environment

Initialize $Q(s, a)$, $Model(s, a)$, for all s, a , and $PQueue$ to empty

Loop forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow policy(S, Q)$
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Model(S, A) \leftarrow R, S'$
- (e) $P \leftarrow |R + \gamma \max_a Q(S', a) - Q(S, A)|.$
- (f) if $P > \theta$, then insert S, A into $PQueue$ with priority P
- (g) Loop repeat n times, while $PQueue$ is not empty:

$S, A \leftarrow first(PQueue)$

$R, S' \leftarrow Model(S, A)$

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

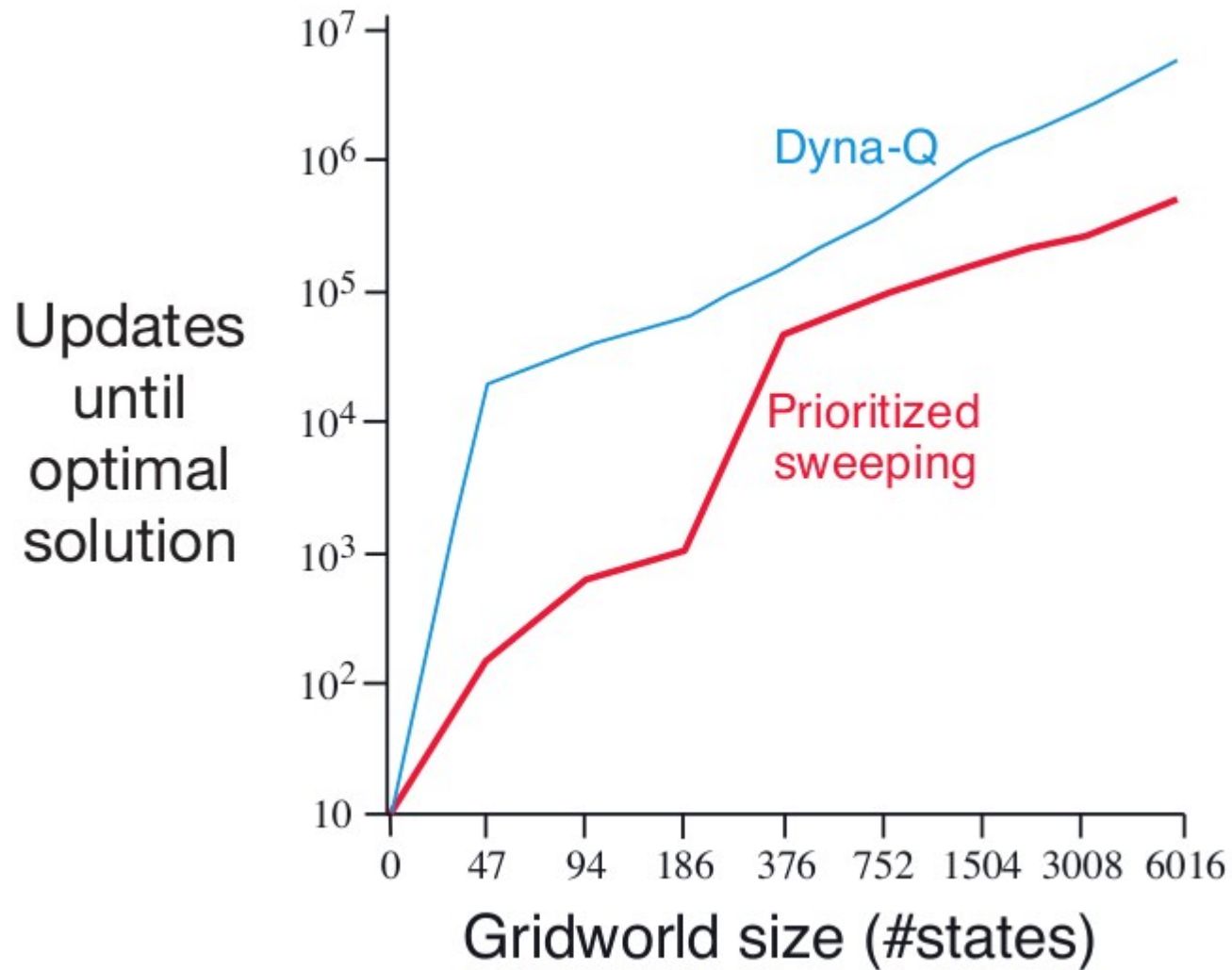
Loop for all $\bar{S}, \bar{A}$ predicted to lead to S :

$\bar{R} \leftarrow$ predicted reward for $\bar{S}, \bar{A}, S$

$P \leftarrow |\bar{R} + \gamma \max_a Q(S, a) - Q(\bar{S}, \bar{A})|.$

if $P > \theta$ then insert $\bar{S}, \bar{A}$ into $PQueue$ with priority P

Example: Prioritized Sweeping of Mazes



Prioritized Sweeping

- Prioritized sweeping works also in **stochastic environments**
- The **model** is maintained by **keeping counts of the # times each state-action has been experienced** and what the **next states** were
- An **expected update** is used, taking into account **all possible next states and their probabilities** of occurring
- **Limitation:** expected updates in stochastic environments may **waste lots of computation** on **low-probability transitions**
- **Solution:** **sample updates** can in many cases get **closer** to the true value function with **less computation** despite the variance introduced

Prioritized Sweeping

- **Sample updates** break the overall backing-up computation into **small pieces** (individual **transitions**) which enable to focus more narrowly on the pieces that have the largest impact
 - **Updates** along a **single transition**
 - Selecting the **order** in which **small updates** are done can **improve planning efficiency beyond** that possible with **prioritized sweeping**
- **Forward focusing:** update states according to the probability to be visited according to the current policy
- **In summary**
 - **Type of update:** expected/sample, large/small
 - **Order of update:** backward/forward

Trajectory Planning

Two ways of distributing updates:

- Sweeps through the **entire state (or state-action) space**: classical approach from **DP**; updating **each state once per sweep**; equal treatment of all states
 - **Problematic on large tasks**. There may not be time to complete even one sweep
 - Updates can be distributed in different ways but in practice exhaustive sweeps are used
- Sample from the state (state-action) space **according to some distribution**
 - **Uniformly** (as in DynaQ): same problems of exhaustive sweep
 - According to **on-policy distribution**: **distribution observed when following the current policy**. Easily generated: **interact with the model following the current policy**. **Trajectory sampling**

Trajectory Sampling

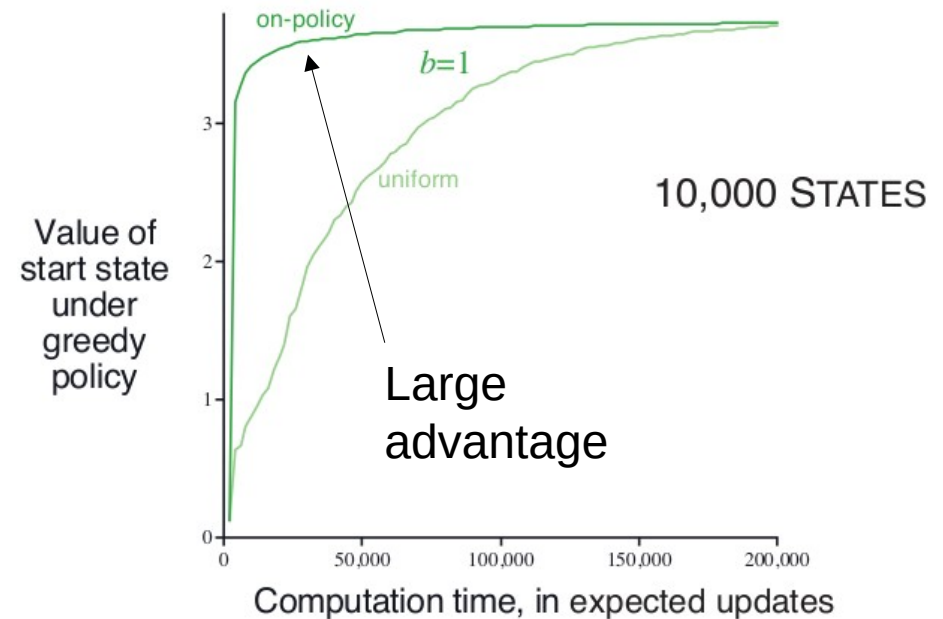
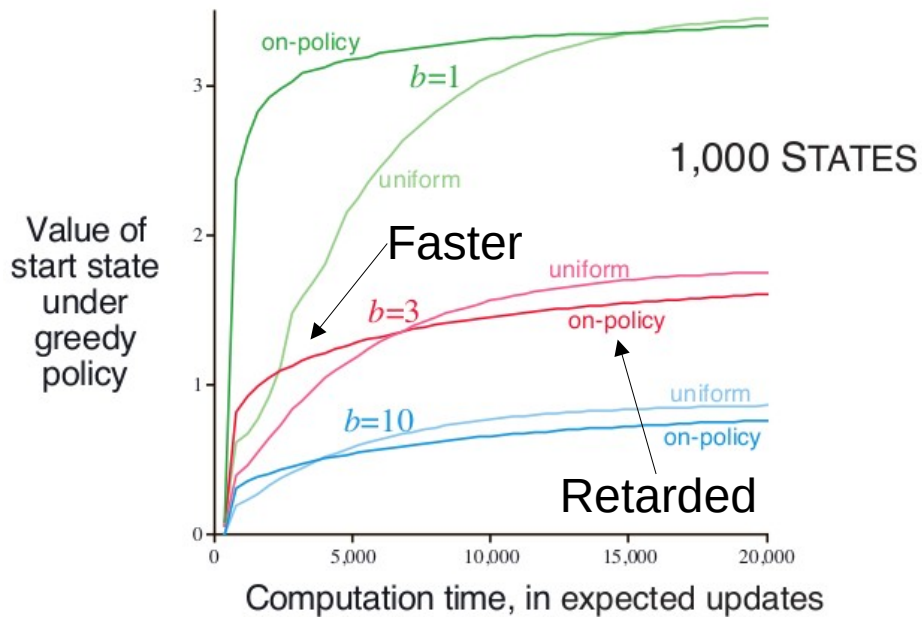
Trajectory sampling: simulate explicit individual trajectories and perform updates at the state (state-action pair) encountered along the way

Question: Is the **on-policy distribution** of updates a **good** one? Yes, at least **better than uniform distribution**

- E.g., if you are learning to play **chess**, you study **positions that might arise in real games**, not random positions of chess pieces
- The latter may be **valid states but less probable, hence less important** for the evaluation of the value function
- On-policy distribution has **significant advantages** when **function approximation** is used (Part II of SutBar)
- In general it achieves **improvement of the speed of planning**

Trajectory Sampling

Experiment to assess the **efficiency of the update distribution:**
uniform vs on-policy



Sampling according to the **on-policy distribution**:

- great **advantage** for **large problems**
- in particular for problems in which a **small subset of the state-action space is visited** under the on-policy distribution

Planning at decision time

Background planning (e.g., DP, Dyna):

- Used to gradually improve a policy on simulated experience
- Played well before an action is selected
- Focused on all states

Decision-time planning

- **Started and completed after encountering a new state S_t**
- The **output** is the **selection of a single action A_t**
- E.g., select the action bringing to (model-predicted) states having the highest value
- **Can look much deeper than one-step ahead**
- **Focused on a particular state**

In the middle

- E.g., focus planning on the current state and store the results

Summary of Part I

- **RL** as a coherent set of **ideas** (idea $\rightarrow$ dimension of method space)
- All methods have 3 ideas in common
 - 1. They all seek to estimate value functions
 - 2. They all backup values along state trajectories
 - 3. They all follow the GPI strategy

Summary of Part I

